

Checkpoint-as-a-Service: Co-designing LLM Training and Storage for Efficient Checkpointing

Participant: Saam (SayedMohammadHossein) Hosseini

1. Project Goals

Large Language Model (LLM) training is one of the most resource-intensive workloads in modern datacenters. During training, models must periodically save their full state — model weights and optimizer states — as **checkpoints**. For production-scale models, a single checkpoint can range from tens of gigabytes to multiple terabytes. Writing these checkpoints to remote storage is a critical bottleneck: while the data is being serialized, compressed, and uploaded, the expensive GPU compute nodes sit **idle**, wasting time and money.

The goal of this project is to build **Checkpoint-as-a-Service (CPaaS)**, an end-to-end checkpoint management service that takes a **co-design approach** between the LLM training layer and the storage infrastructure layer. Rather than optimizing either side in isolation (as most existing work does), CPaaS enables the two layers to communicate and adapt to each other in real time. The service will:

1. **Minimize checkpoint stall time** — reduce the wall-clock time that training nodes are blocked waiting for checkpoint I/O to complete.
2. **Adaptively compress checkpoint data** — dynamically enable or disable client-side compression (e.g., ZSTD) based on real-time storage cluster load, so compression is only used when it provides a net benefit.
3. **Balance upload load across storage nodes** — use adaptive load balancing to route checkpoint traffic away from overloaded storage nodes.
4. **Redistribute checkpoint chunks across training nodes** — enable peer-to-peer chunk redistribution so that all training nodes upload a balanced amount of data, eliminating bottlenecks caused by uneven checkpoint sizes.

Success criteria: Measurable reduction in average checkpoint time (seconds) and improvement in upload throughput (MB/s) compared to a naive baseline, across multiple realistic scenarios involving storage node stress and data imbalance.

2. Software and Hardware Components

2.1 Hardware

Component	Role	Specs
1× Load Balancer / Control Plane (sam-lb)	NGINX reverse proxy, control plane coordinator	e2-medium, 2 vCPU, 4 GB RAM
4× Storage Nodes (minio1–minio4)	Distributed object storage cluster	e2-standard-4, 4 vCPU, 15 GB RAM, 200 GB SSD each

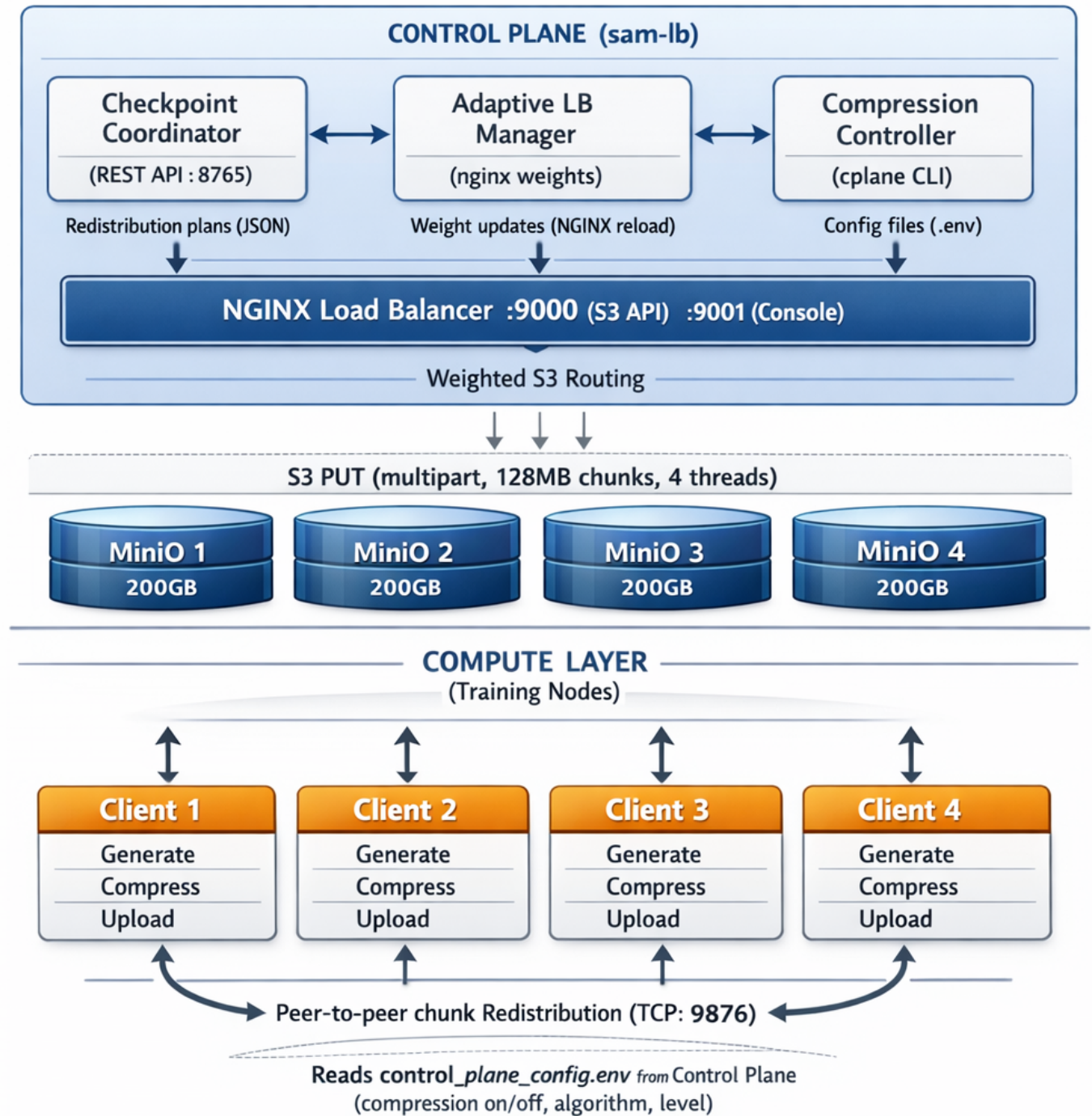
Component	Role	Specs
4× Client/Compute Nodes (client1–client4)	Simulate distributed training, generate and upload checkpoints	n2-standard-2, 2 vCPU, 8 GB RAM, 250 GB SSD each

Total: 9 GCP Virtual Machines

2.2 Software

Component	Technology	Role
Distributed Object Storage	MinIO (S3-compatible, 4-node erasure-coded cluster)	Persistent checkpoint storage backend
Load Balancer	NGINX (reverse proxy, <code>least_conn</code>)	Routes S3 API traffic to storage nodes
Checkpoint Script	Python (PyTorch, Safetensors, boto3)	Generates, compresses, and uploads checkpoint data
Checkpoint Coordinator	Python HTTP API (port 8765)	Central coordinator for redistribution planning
Chunk Transfer System	Custom TCP server/client (port 9876)	Peer-to-peer chunk redistribution between training nodes
Adaptive LB Manager	Bash (<code>nginx-lb-manager.sh</code>)	Adjusts NGINX weights based on per-node CPU utilization
Control Plane CLI	Bash (<code>cplane</code>)	Manages compression policies, metrics, and experiments
Compression	ZSTD, S2/Snappy, GZIP	Client-side checkpoint compression
Metrics Collection	SSH-based scripts + Prometheus	System, I/O, and network metrics from all nodes
Container Runtime	Docker	Packaging and deployment of services

3. Architectural Diagram



Data Flow

- | | |
|-----------------|--|
| 1. GENERATE | Client generates checkpoint (model.safetensors + optimizer.pt) |
| | |
| 2. COORDINATE | Client calls POST /checkpoint/start → Coordinator returns redistribution plan (who sends what chunks to whom) |
| | |
| 3. REDISTRIBUTE | Clients split files into 64MB chunks and transfer excess chunks to peers via TCP (:9876) to balance upload sizes |
| | |
| 4. COMPRESS | Client reads control_plane_config.env → decides whether to compress (ZSTD/S2/GZIP) based on cluster state |
| | |
| 5. UPLOAD | Client uploads kept + received chunks to MinIO via NGINX LB using S3 multipart upload (128MB parts, 4 threads) |
| | |
| 6. REPORT | Client calls POST /checkpoint/complete → Coordinator learns throughput via EMA, computes next plan |
| | |
| 7. ADAPT | LB Manager polls MinIO CPU metrics → adjusts NGINX weights → routes traffic away from hot nodes |
-

4. Component Interaction Description

4.1 Training Layer → Storage Layer (Checkpoint Upload)

Each client node simulates a distributed training rank using PyTorch. At each checkpoint step, it serializes the model weights as a `.safetensors` file (~1 GB for a 0.5B model) and optimizer states as a `.pt` file (~5.9 GB). These files are uploaded to the MinIO cluster via the NGINX load balancer using the S3 `put_object` API with multipart uploads (128 MB chunks, 4 parallel threads).

4.2 Control Plane → Training Layer (Adaptive Compression)

The control plane daemon on `sam-lb` continuously monitors MinIO node CPU utilization via SSH. When the average CPU exceeds 80%, it writes `COMPRESSION_ENABLED=true` to `/opt/checkpointing/control_plane_config.env` on each client. When CPU drops below 50%, it disables compression. Each client reads this file before every upload, dynamically toggling ZSTD compression. This co-design ensures compression is only applied when the storage cluster is under pressure and the compute nodes can absorb the CPU cost.

4.3 Control Plane → Storage Layer (Adaptive Load Balancing)

The adaptive LB manager polls each MinIO node's CPU and computes an NGINX weight using: $\text{weight} = \max(1, \min(10, \text{round}(10 \times (1 - \text{cpu}/100))))$. A node at 90%+ CPU gets weight 0 (marked down). The NGINX config is rewritten and reloaded, steering new checkpoint traffic away from overloaded nodes and toward healthy ones.

4.4 Training Layer Training Layer (Peer-to-Peer Redistribution)

When checkpoint sizes are imbalanced across training ranks (e.g., Client1 has 800 MB, Client2 has 200 MB), the Checkpoint Coordinator computes a redistribution plan to equalize upload sizes. Clients split their files into 64 MB chunks and transfer excess chunks to peers over TCP (port 9876). Each client then uploads only its assigned portion — both locally-kept chunks and received chunks — to MinIO. This eliminates the “slowest client” bottleneck. Experiments show a **~19% reduction** in checkpoint time with the simple size-proportional strategy.

4.5 Metrics Pipeline

SSH-based metric collection scripts gather CPU, memory, disk I/O, and network stats from all 9 nodes. These metrics feed the control plane’s decision logic (compression toggling, LB weight updates) and are logged for post-experiment analysis.

5. Debugging and Testing Plan

5.1 Simulation Mode

The checkpoint script includes a full **simulation mode** that generates realistic but synthetic checkpoint data (with sparsity, quantization patterns, and repeated blocks) without requiring GPUs or loading real models. This allows rapid iteration and testing of the entire pipeline on CPU-only VMs.

5.2 Component-Level Testing

Component	Test Method
Chunk Transfer (TCP)	Verify MD5 checksums on received chunks; test with partial sends and network interruptions
Coordinator API	Unit tests against the REST API; verify redistribution plans produce balanced assignments
Compression	Compare compressed vs. original file sizes and verify decompression correctness
NGINX LB	Verify weight updates take effect by checking <code>nginx -T</code> output and observing traffic distribution
MinIO Cluster	Verify erasure coding with <code>mc admin info</code> ; test object GET after node failure

5.3 End-to-End Experiment Framework

The project includes an automated experiment framework (`lb-experiment.sh`, `strategy_comparison` scripts) that runs controlled, multi-phase experiments:

1. **Baseline phase:** All nodes healthy, equal weights, no compression — establishes reference metrics.
2. **Stress phase:** Inject CPU load on selected MinIO nodes using `stress-ng`, measure degradation.

3. **Adaptive phase:** Enable co-design features (adaptive LB, compression, redistribution), measure improvement.

Each phase records checkpoint time, throughput, per-node CPU/memory/disk/network metrics, and compression ratios. Results are saved to structured output directories for analysis.

5.4 Logging and Observability

- Every checkpoint upload logs whether compression was controlled by the **control plane override** or **default settings**, with timestamps.
- The coordinator logs redistribution plans, EMA-learned weights, and per-client throughput after each step.
- The control plane daemon logs all decisions and cooldown events.

6. Cloud Technologies Justification

This project uses the following datacenter software components, satisfying the requirement of at least four:

#	Required Component	Implementation in CPaaS
1	RPC / API Interfaces	REST API on the Checkpoint Coordinator (HTTP/JSON on port 8765) for redistribution plan negotiation; S3 API for all storage operations
2	Storage Services (S3, etc.)	MinIO distributed object storage cluster (4-node, erasure-coded, S3-compatible)
3	Virtual Machines, Containers	9 GCP VMs running the full distributed system; Docker containers for service packaging
4	Key-Value Stores	Control plane config files (.env) act as a lightweight distributed key-value mechanism for real-time policy propagation; checkpoint metadata tracking in the coordinator
5	Message Marshalling / Encoding	JSON message encoding for coordinator API requests/responses; custom binary protocol with 4-byte header + JSON metadata + raw bytes for TCP chunk transfer
6	Software Defined Networks	GCP VPC internal networking with NGINX reverse proxy performing adaptive, metric-driven load balancing across storage nodes

This project goes well beyond “using an existing tool” — it builds a complete, multi-component distributed service with real-time inter-layer coordination, adaptive policies, and a custom peer-to-peer data transfer protocol.